



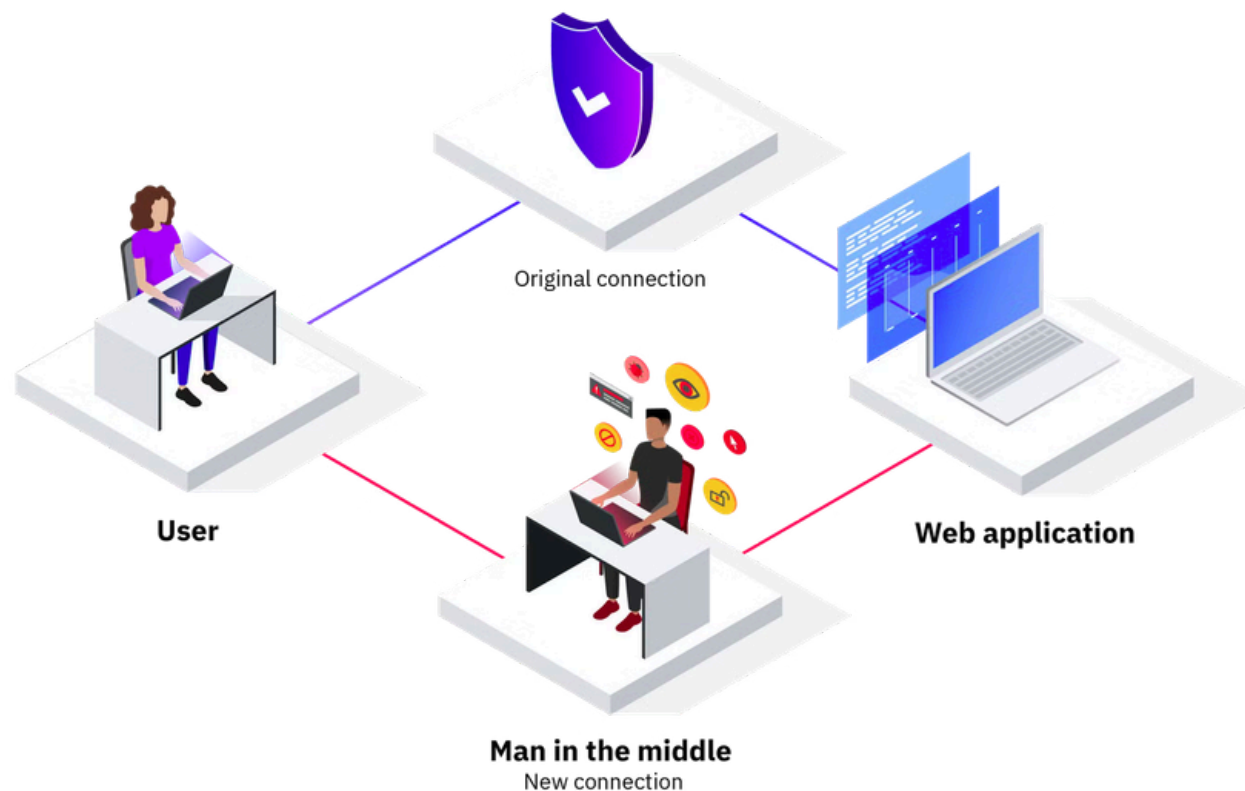
UNIVERSIDADE FEDERAL DO ESPÍRITO SANTO

## Controle de ataques MITM com Falco

Luca da Silva Ávila

# O Problema

Man-in-the-Middle é um ataque que uma terceira parte consegue se infiltrar na comunicação entre duas partes



# O Problema

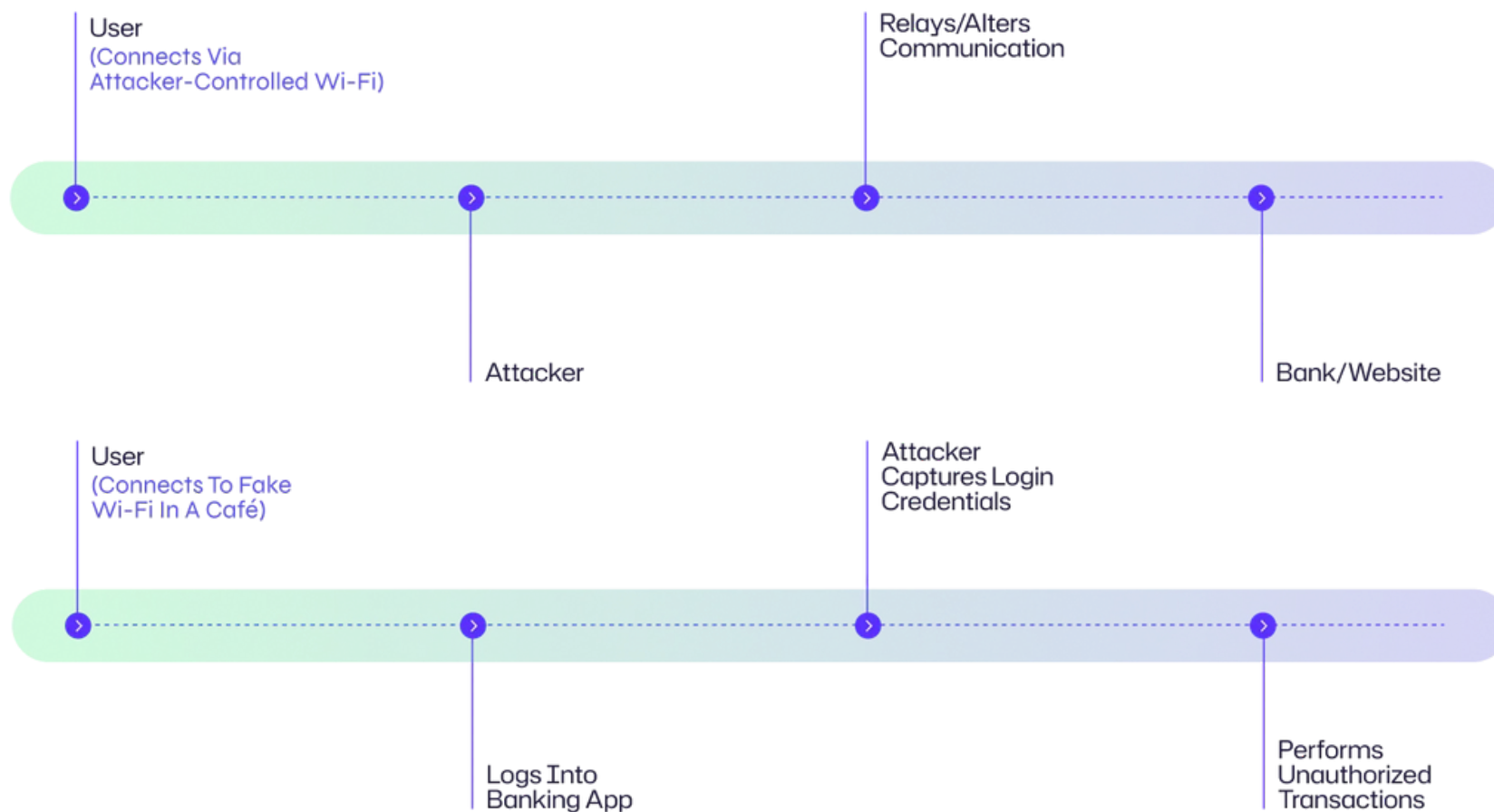
Como é feito um ataque MITM?

**Interceptação:** Esta é a primeira fase, onde os atacantes se posicionam secretamente entre as duas partes que estão se comunicando, como inserir um dispositivo de escuta nas linhas telefônicas sem que ninguém perceba e depois ouvir a conversa de duas ou mais partes.

**Descriptografia:** Uma vez que um atacante coletou dados após a interceptação da comunicação, o próximo passo é decifrar esses dados, especialmente se a comunicação for criptografada (sobre o protocolo https), pois dados criptografados são apenas um amontoado de caracteres sem sentido sem a descriptografia.

# O Problema

## Fluxo de Ataque MITM



# O Problema

Técnicas para interceptar e descriptografar dados durante ataques

**Falsificação IP**

**Falsificação  
de DNS**

**Sequestro de SSL**

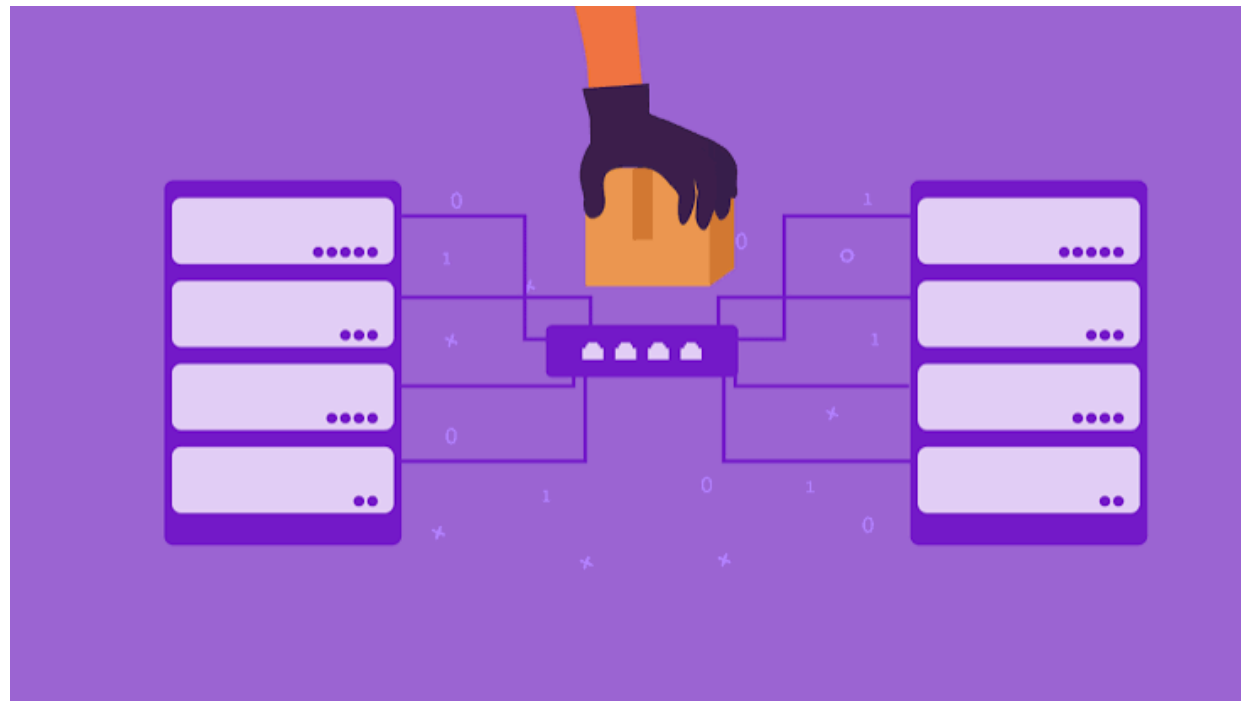
**Falsificação de ARP ou  
envenenamento de  
cache ARP**

**Falsificação de HTTPS**

**Eliminação de SSL**

# O Problema

Isso não é fácil de perceber



# O Código

## Criação de Ambiente para Ataques Man-in-the-Middle

- Docker-compose para ambiente
- Três abas abertas no container do atacante
- Uma aba aberta no container do cliente
- Uma aba aberta no container do servidor

# O Código

## Criação de Ambiente para Ataques Man-in-the-Middle

- Docker-compose para ambiente

```
services:
  victim:
    image: handsonsecurity/seed-ubuntu:large
    container_name: victim-10.9.0.5
    tty: true
    networks:
      mitm-net:
        ipv4_address: 10.9.0.5
```

```
server:
  image: handsonsecurity/seed-ubuntu:large
  container_name: server-10.9.0.6
  tty: true
  networks:
    mitm-net:
      ipv4_address: 10.9.0.6
```

```
attacker:
  image: nicolaka/netshoot # Imagem com arpspoof nativo
  container_name: attacker-10.9.0.99
  tty: true
  privileged: true
  networks:
    mitm-net:
      ipv4_address: 10.9.0.99
```



# O Código

## Criação de Ambiente para Ataques Man-in-the-Middle

- Container de Atacante

**Aba 1:**

```
docker exec -it attacker-10.9.0.99 bash
```

```
sysctl -w net.ipv4.ip_forward=1
```

```
arp spoof -i eth0 -t 10.9.0.5 10.9.0.6
```

**Aba 2:**

```
arp spoof -i eth0 -t 10.9.0.6 10.9.0.5
```

**Aba 3:**

```
tcpdump -i eth0 -A
```

# O Código

## Criação de Ambiente para Ataques Man-in-the-Middle

- Container de Vítima

```
docker exec -it victim-10.9.0.5 bash
```

```
echo "Sua senha super secreta de teste" | nc 10.9.0.6 9000
```

- Container de Servidor

```
docker exec -it server-10.9.0.6 nc -l -p 9000
```

# O Código

## Execução de Ataque MITM

### Vítima

```
root@47a31ab85e03:/# echo "Sua senha super secreta" | nc 10.9.0.6 9000
```

### Servidor

```
root@7f5cd6aa5478:/# nc -l -p 9000  
Sua senha super secreta
```

### Atacante

```
04:57:37.484042 IP victim-10.9.0.5.oexperimento_mitm-net.38006 > server-10.9.0.6.oexperimento_mitm-net.9000: Flags [P.], seq 1:25, ack 1, win 502, options [nop,nop,TS val 2166960884 ecr 768767291], length 24  
E..L6.@.?...  
..  
...v#(.PHMq...-.....[.....  
)2.-.u;Sua senha super secreta
```

# O Código

## Aplicação do Falco no projeto

`docker logs -f falco-monitor`

```
falco:
  image: falcosecurity/falco:latest
  container_name: falco
  privileged: true
  environment:
    - FALCO_STDOUT_OUTPUT_ENABLED=true
    - FALCO_JSON_OUTPUT=false
    - FALCO_MODERN_EBPF_TOCTOU_MITIGATION_ENABLED=false
  volumes:
    - /var/run/docker.sock:/var/run/docker.sock
    - /dev:/host/dev
    - /proc:/host/proc:ro
    - /boot:/host/boot:ro
    - /lib/modules:/host/lib/modules:ro
    - ./custom_rules.yaml:/etc/falco/rules.d/custom_rules.yaml:ro
  command: ["/usr/bin/falco"]
  restart: always
```

# O Código

## Aplicação de Monitoramento de MITM - Falco

```
def kill_process_by_name(process_name, contextual_message="🛑 ANULAÇÃO"):
    """Localiza o processo no host e o encerra imediatamente."""
    try:
        result = subprocess.run(["pgrep", "-f", process_name], stdout=subprocess.PIPE, text=True)
        pids = result.stdout.strip().split("\n")

        killed_any = False
        for pid_str in pids:
            if pid_str:
                pid = int(pid_str)
                if pid != os.getpid(): # Evita suicídio do script Python
                    os.kill(pid, signal.SIGKILL)
                    print(f"↳ {contextual_message}: PID {pid} ({process_name}) foi neutralizado.")
                    killed_any = True
        return killed_any
    except Exception as e:
        print(f"↳ ⚠ Erro ao tentar encerrar processo: {e}")
        return False
```

# O Código

## Aplicação de Monitoramento de MITM - Falco

```
def monitor_falco_preventivo():
    print("🛡️ [SISTEMA DE MITIGAÇÃO PROATIVA] Ativado.")

    # --- PASSO NOVO: SANITIZAÇÃO INICIAL ---
    print("🔍 [VARREDURA] Procurando por ataques ativos que já estavam rodando...")
    houve_limpeza = kill_process_by_name("arpspoof", contextual_message="✓ LIMPEZA INICIAL")
    if not houve_limpeza:
        print("👉 Nenhum ataque pré-existente encontrado. Sistema limpo.")

    print("\n🚀 Iniciando monitoramento em tempo real via Falco (Pré-Rede)...")
    print("💡 Aguardando novas tentativas de execução...\n")

    # stdbuf -oL impede o buffer de bloco; --tail 0 ignora o passado no log do Falco
    cmd = ["stdbuf", "-oL", "docker", "logs", "-f", "--tail", "0", "falco"]

    process = subprocess.Popen(
        cmd,
        stdout=subprocess.PIPE,
        stderr=subprocess.PIPE,
        text=True
    )

    for line in iter(process.stdout.readline, ""):
        line_lower = line.lower()

        # Captura novas inicializações do binário malicioso
        if "arpspoof" in line_lower and ("execve" in line_lower or "executing binary" in line_lower):

            print(f"\n⚠️ [ALERTA DE PRÉ-ATAQUE DETECTADO VIA FALCO]:")
            print(f"👉 {line.strip()[:130]}...")
            print(f"🚨 [REAÇÃO PREVENTIVA]: Tentativa de nova execução interceptada!")

            sucesso = kill_process_by_name("arpspoof", contextual_message="🛑 BLOQUEIO EM TEMPO REAL")

            if not sucesso:
                # Margem de segurança de milissegundos para o escalonador do Kernel
                time.sleep(0.01)
                kill_process_by_name("arpspoof", contextual_message="🛑 BLOQUEIO EM TEMPO REAL (REINSPEÇÃO)")

            print("-" * 85)
            sys.stdout.flush()

if __name__ == "__main__":
    try:
        monitor_falco_preventivo()
    except KeyboardInterrupt:
        print("\n🛑 Monitoramento preventivo encerrado.")
```

# O Código

## Aplicação de Monitoramento de MITM - Código eBPF

```
#!/usr/bin/env python3
from bcc import BPF
import ctypes

# Código C que usa Tracepoints estáveis do Kernel
ebpf_code = """
#include <linux/sched.h>

struct data_t {
    u32 pid;
    char comm[TASK_COMM_LEN];
    u32 family;
};

BPF_PERF_OUTPUT(events);

// Hook estável na entrada da syscall socket
TRACEPOINT_PROBE(syscalls, sys_enter_socket) {
    struct data_t data = {};

    // args->family extrai o valor diretamente da definição de tracepoint do kernel
    int family = args->family;

    // AF_PACKET (constante 17)
    if (family == 17) {
        data.pid = bpf_get_current_pid_tgid() >> 32;
        bpf_get_current_comm(&data.comm, sizeof(data.comm));
        data.family = family;

        events.perf_submit(args, &data, sizeof(data));
    }
    return 0;
}
"""
```

```
class EventData(ctypes.Structure):
    _fields_ = [
        ("pid", ctypes.c_uint32),
        ("comm", ctypes.c_char * 16),
        ("family", ctypes.c_uint32)
    ]

print("🛡️ Iniciando detector de MITM via eBPF Tracepoint...")
b = BPF(text=ebpf_code)

def print_event(cpu, data, size):
    # Correção: POINTER em maiúsculas
    event = ctypes.cast(data, ctypes.POINTER(EventData)).contents
    process_name = event.comm.decode('utf-8', 'replace')

    print(f"\n⚠️ [ALERTA DE SEGURANÇA eBPF]")
    print(f"  ↳ Detecção: Tentativa de falsificação de rede / MITM")
    print(f"  ↳ Processo: {process_name}")
    print(f"  ↳ PID no Host: {event.pid}")
    print(f"  ↳ Família: AF_PACKET (Acesso de rede de baixo nível)")
    print("-" * 60)

# O BCC mapeia automaticamente o tracepoint
b["events"].open_perf_buffer(print_event)
while True:
    try:
        b.perf_buffer_poll()
    except KeyboardInterrupt:
        print("\nDetector encerrado.")
        break
```

# O Código

## Código eBPF simples

Ancoragem feita em tracepoint `sys_enter_socket`  
Perf buffer para envio de dados ao user-space

## Falco

Ancoragem feita em `syscall` (syscall table)  
Ring buffer para envio de dados ao user-space



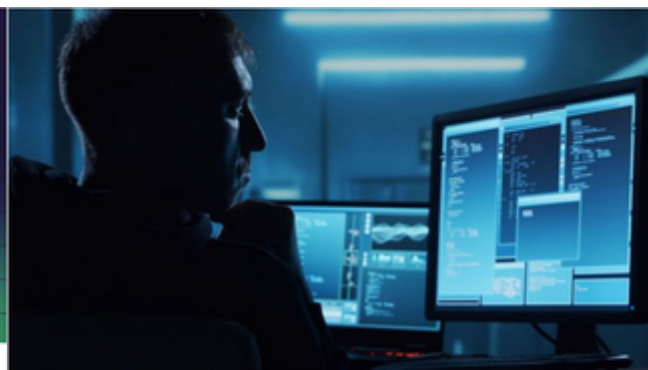
# OBRIGADO

# netwrix

## Ataques Man-in-the-Middle (MITM): Riscos, Detecção & Proteção

Aprenda como funcionam os ataques Man in the Middle, exemplos reais e estratégias eficazes para prevenir ameaças MITM.

 Netwrix / May 26



## O que é um ataque intermediário (MITM)?

Um ataque intermediário (MITM) é um cibercrime em que um hacker rouba informações sensíveis ao interceptar comunicações entre dois alvos online.

 ibm.com / Nov 11, 2025



Detect  
**CVE-2020-8554**  
using Falco



## Detect CVE-2020-8554 – Unpatched Man-In-The-Middle (MITM) Attack in Kubernetes

How to detect the CVE-2020-8554 vulnerability that allows users to intercept traffic from other pods or nodes in a Kubernetes cluster.

 Sysdig / Mar 27

### kientuong114/ docker-mitm

A Man in the Middle proof of concept using docker containers

1 Contributor 1 Issue 21 Stars 7 Forks



kientuong114/docker-mitm: A Man in the Middle proof of concept using docker containers

A Man in the Middle proof of concept using docker containers - kientuong114/docker-mitm

 GitHub

# TALES FROM THE KERNEL PARAMETER SIDE

## Tales from the Kernel Parameter Side

Armed with knowledge about what these kernel parameters are, we can work on putting them to use in strengthening the security of our systems.

 Sysdig / Mar 27